Hazelnut Survey Data Architecture

This document captures how the current Hazelnut dashboard expects survey data to look once it reaches analytics. It is written as a practical reference for building a new survey system (for example, for Truffle) while keeping compatibility with the existing dashboard approach.

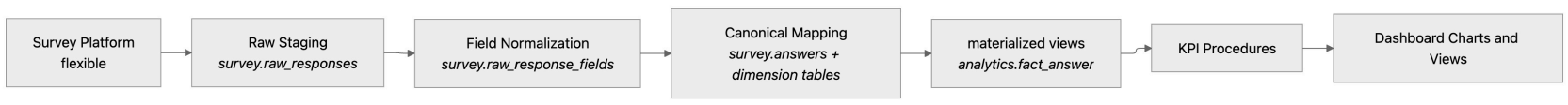
The key idea in the current architecture is that KPI logic does not read directly from provider payloads. It reads from a normalized analytics entry point: `analytics.fact_answers` (materialized view), which is built from selected `survey` schema tables.

1. System Overview

The Hazelnut implementation separates collection, normalization, and analytics:

- Survey responses are collected externally (currently Qualtrics).
- Ingestion functions normalize the responses into canonical `survey` tables.
- A materialized fact layer, `analytics.fact_answers`, is refreshed from canonical survey answers.
- KPI calculations are based on that fact layer, and dashboard charts read KPI outputs.

2. Architecture Diagram



In the current system, `fact_answers` is the bridge between survey ingestion and analytics/KPI logic.

3. Database Schema (Compatibility Scope)

The most relevant schema scope is the portion that feeds `fact_answers`:

- `survey.answers`: normalized answer facts; supports numeric/text/boolean and optional row/column/option references.
- `survey.surveys`: provider survey identity and version metadata.
- `survey.questions`: maps the internal ID to the canonical question code.
- `survey.question_rows`: maps the internal ID to the canonical row code and row label.
- `survey.question_columns`: maps the internal ID to the canonical column code and column label.
- `survey.question_options`: maps the internal ID to the canonical option code and option label.

Supporting ingestion and mapping context:

- `survey.raw_responses`: full raw provider payload per provider response ID.
- `survey.raw_response_fields`: decomposed/staged fields parsed from `payload.values` and `payload.labels` (qid, row_code, column_code, suffix, value_json, label_json) used as the intermediate normalization input before mapping into canonical answers.
- `survey.provider_mapping`: maps provider-specific field codes (`provider_qid`, row/column/option/suffix) to canonical question/row/column/option codes.
- `survey.respondents`: person-level entity; links to auth where available.
- `survey.farms`: farm entity tied to respondent.

Row/Column data for matrix questions

In this project, `row` and `column` data mainly exist to represent matrix-style survey questions in a canonical way.

- `survey.question_rows` stores row dimension metadata for a question (row code and row label).
- `survey.question_columns` stores column dimension metadata for a question (column code and column label).

- `survey.answers.row_id` and `survey.answers.column_id` reference those dimensions when a response comes from a matrix cell.

For non-matrix questions, `row_id` and `column_id` are usually `NULL`. For matrix questions, they preserve the full cell context:

- which row item the response belongs to
- which column scale or attribute the response belongs to

This context flows into `analytics.fact_answers` as `row_code` and `column_code`, allowing KPI calculations to aggregate at matrix-cell level rather than flattening away matrix structure.

4. Table Relationships

From a fact-building perspective, relationships are straightforward:

- `survey.answers.response_id -> survey.survey_responses.id`
- `survey.answers.question_id -> survey.questions.id`
- `survey.answers.row_id -> survey.question_rows.id` (optional)
- `survey.answers.column_id -> survey.question_columns.id` (optional)
- `survey.answers.option_id -> survey.question_options.id` (optional)

The `fact_answers` view then combines these links into a query-friendly, analysis-ready shape with:

- identifiers (`respondent_id`, `farm_id`, `year`, `response_id`)
- canonical coordinates (`question_code`, `row_code`, `column_code`)
- either numeric value (`value_number`) or option value (`option_code`, `option_label`)

5. Data Contract (Required Fields)

Seen from the analytics boundary, the data contract is mostly about what must exist to build `fact_answers` reliably.

Core identity fields:

- stable submission identifier (`response_id`)
- respondent linkage (`respondent_id`)
- farm linkage (`farm_id`)
- benchmarking year (`year`)

Canonical answer fields:

- `answers.question_id`
- optional matrix coordinates (`row_id`, `column_id`)
- optional option pointer (`option_id`)
- typed payload (`value_number` and/or option linkage; optional `other_text`)

Canonical metadata fields:

- `questions.code`
- optional `question_rows.code`, `question_columns.code`
- optional `question_options.code`, `question_options.label`

The compatibility value comes from stable canonical codes and repeatable mapping, not from any specific source-provider key format.